FemAI: Project Handover & Documentation

Project Title: FemAI - An Intelligent PCOS Screening and Lifestyle Guidance Platform

Course: AI/ML Honors Mini-Project

Domain: Healthcare & Artificial Intelligence

🔗 Repository Link

GitHub Repo: https://github.com/NishigandhaSawant/FemAI.git

📖 About the Project

Polycystic Ovary Syndrome (PCOS) affects roughly 1 in 10 women globally but remains highly underdiagnosed due to complex and overlapping symptoms. **FemAI** is an intelligent, privacy-centric web platform designed to bridge the gap between clinical data and accessible early detection.

Unlike traditional "black-box" ML projects that simply output "Positive/Negative", FemAI utilizes **SHAP (Explainable AI)** to tell the user *why* they were flagged (e.g., elevated BMI or irregular cycle length) and uses an automated logic engine to provide immediate, actionable lifestyle and dietary guidance based on their specific risk profile.

🎯 Official Project Objectives

1. **Algorithm Comparative Analysis:** To evaluate and compare multiple machine learning classifiers (Logistic Regression, Random Forest, XGBoost) on high-dimensional clinical data to establish the most robust model for PCOS screening.
2. **Model Explainability & Optimization:** To enhance system transparency using explainable AI (SHAP) and handle medical data imbalance using SMOTE, identifying the primary physiological markers that drive predictions.
3. **Automated Lifestyle Guidance:** To develop a recommendation engine that maps the specific risk factors flagged by the ML model to personalized dietary and fitness interventions.
4. **Responsive System Integration:** To design and deploy a secure, full-stack web application (React + FastAPI) that serves the predictive models through a user-friendly interface.

🏗️ Planning & Architecture

The project follows a modern 3-Tier Architecture:

1. **Client Tier (Frontend):**
    - **Tech:** React.js (Vite), Tailwind CSS, Axios.
    - **Role:** Captures physical/clinical metrics via a responsive form and displays the diagnosis, SHAP insights, and lifestyle action plan.
2. **Application Tier (Backend):**
    - **Tech:** Python, FastAPI, Uvicorn, Pydantic.
    - **Role:** Exposes a RESTful API (/predict). It receives the frontend data, scales it using a pre-fitted StandardScaler, and manages the flow of data.
3. **Intelligence Tier (Machine Learning):**
    - **Tech:** Scikit-Learn, XGBoost, SHAP, Imbalanced-learn (SMOTE).
    - **Role:** The trained and pickled (.pkl) models ingest the scaled data to predict PCOS probability and extract feature importance.

✅ Progress: What We Have Done Till Now

We have successfully completed the Data Science pipeline and the Backend engine. We are currently at the Frontend integration phase.

1. Data Science & ML Phase (Completed)

- Identified that the standard 541-row dataset was too small and prone to overfitting. **Upgraded to a 2000-row extended clinical dataset** for better generalization.
- Handled the extreme class imbalance in the medical data by applying **SMOTE**, perfectly balancing positive and negative cases for training.
- Ran a comparative analysis between Logistic Regression, Random Forest, and XGBoost.
- Implemented **SHAP** to extract feature importance (identifying Follicle Count, Cycle Length, and AMH as top indicators).
- Exported the winning model (pcos_model.pkl) and scaler (scaler.pkl).

2. Backend Engineering Phase (Completed)

- Initialized an isolated Python virtual environment (venv).
- Built a lightning-fast API using **FastAPI** (main.py).
- Configured **CORS Middleware** so the future React frontend can communicate with the server without security blocks.
- Created the /predict POST endpoint to ingest JSON payloads, process them through the .pkl files, and return diagnosis booleans and risk percentages.
- *Bug Fix:* Resolved a critical NumPy 2.0 vs Pandas dependency clash by strictly downgrading to "numpy<2". The server now boots flawlessly.

3. Frontend Setup Phase (In Progress)

- Scaffolded a blazing-fast React environment using **Vite**.
- Selected **ESLint** as the linter to ensure code quality during collaborative AI generation.
- Installed essential UI dependencies: tailwindcss, axios, and lucide-react (for icons).

📂 Project File Structure

Here is exactly how our local workspace is organized. **Keep the backend and frontend folders strictly separated.**

```
femai/
├── backend/
│   ├── venv/              # Isolated Python environment (do not commit to git)
│   ├── main.py            # The FastAPI server code
│   ├── pcos_model.pkl     # Our trained machine learning model
│   ├── scaler.pkl         # Fitted scaler for standardizing user input
│   └── requirements.txt   # List of backend dependencies
│
├── frontend/
│   ├── node_modules/      # React dependencies
│   ├── src/
│   │   ├── App.jsx        # Main React component (UI will be built here)
│   │   ├── main.jsx       # React entry point
│   │   └── index.css      # Tailwind CSS injections
│   ├── package.json       # Frontend dependency list
│   ├── tailwind.config.js # Tailwind design tokens
│   └── vite.config.js     # Vite bundler config
│
└── research/
    ├── colab_notebook.ipynb  # The ML training and graphing code (for Viva/Professor)
    └── PCOS_extended_data.csv # The 2000-row dataset used for training
```

🚀 Next Steps (Action Items for the Team)

1. **Frontend "Vibe Coding":** We need to use an AI IDE (like Windsurf/Cursor) to generate the App.jsx UI using Tailwind CSS. It needs a clinical assessment form, a results dashboard, and a lifestyle guidance component.
2. **API Connection:** Ensure the Axios POST request in the frontend maps perfectly to http://127.0.0.1:8000/predict.
3. **Presentation Deck:** Transfer the academic content from this document into the required PPT template.